CSE344 HW2 REPORT

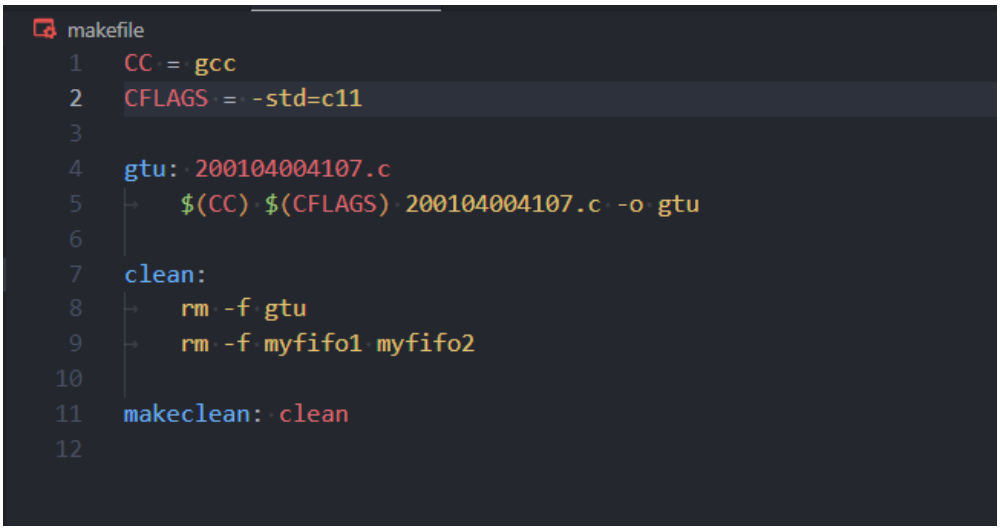
200104004107 Veysel Cemaloğlu

I accomplished the tasks in the assignment including the bonus parts.

You can compile my code by typing “make” in terminal. (makefile is inside the zip file)

You can run by typing “./gtu <array size>“ which array size can only be positive numbers. For example “./gtu 5” will create an arry of 5 random integers and the program will work accordingly to that.

MAKEFILE



You can see the output below. The first 5 proceeding texts were printed every two seconds for the children who waited for 10 seconds.

Then created array is sending to fifos. And child processes starts doing their tasks.

The summation result is 19 which is printed. And this is send to fifo2.

Second child reads the summation result and prints it to screen to confirm they are same.
Multiplication result is calculated and printed to screen which is 243.
And summation of each processes result is 19 + 243 = 262 is printed to screen.

“The signal handler should call waitpid() to reap the terminated child process, print out the process ID of the exited child, and increment a counter.” Here you can see the output of this task. Whenever a child is terminated it will print the info related to that child.

Lastly the parent process is done and the program exits.

```
veysel@LAPTOP-28PS531T:~/projects/cse344$ ./gtu
Usage: ./gtu <array size> (non-negative)
veysel@LAPTOP-28PS531T:~/projects/cse344$ ./gtu 5
Array: 9 3 3 1 3
Proceeding...
Proceeding...
Proceeding...
Proceeding...
Proceeding...
Array sent to FIFO2
Array sent to FIFO1
Proceeding...
Sum of elements in array at CHLD1: 19
Sum sent to CHLD2: 19
Child with PID 14808 terminated. exit status: 0
Proceeding...
Sum from CHLD1 : 19
Result of multiplication: 243
result of summation + result of mult: 262
Child with PID 14809 terminated. exit status: 0
Parent exits
```

If the program is run to complete without stopping, it automatically deletes the fifo files. However, if you close the program with Ctrl+C while the program is running, you need to delete the fifos with “make clean”. It also removes the object file.

This is my signal handler which is right below:

```c
void child_exit_handler(int sig) {
    int status;
    pid_t pid;
    while ((pid = waitpid(-1, &status, WNOHANG)) > 0) {
        if(WIFEXITED(status)){
        printf("Child with PID %d terminated. exit status: %d\n", pid, WEXITSTATUS(status));
        }
        proc_count++;
    }
}
```

I implemented bonus part 2.
which is “2. Print the exit statuses of all processes for an additional 15 points.”
at the line where printf is, you can see that it will print related info to that process.

Inside the signal handler function, the waitpid function is called with the WNOHANG flag, which makes it non-blocking. This allows the parent process to continue executing other tasks while waiting for child processes to terminate.

```
struct sigaction sa;
sa.sa_handler = child_exit_handler;
sa.sa_flags = SA_RESTART;
sigemptyset(&sa.sa_mask);

if (sigaction(SIGCHLD, &sa, NULL) == -1) {
    perror("sigaction");
    exit(EXIT_FAILURE);
}
```

sigaction function is used to set up the signal handler for the SIGCHLD signal. If sigaction fails, an error message is printed, and the program exits with a failure status.

I implemented error scenarios. For example for fifos:
You can see that program checks if fifos created properly, if not it prints related error messages.

```
if(mkfifo(FIFO1, 0666) == -1) {
    if(errno != EEXIST) {
        perror("FIFO1");
        exit(EXIT_FAILURE);
    }
}
if(mkfifo(FIFO2, 0666) == -1) {
    if(errno != EEXIST) {
        perror("FIFO2");
        exit(EXIT_FAILURE);
    }
}
```

I created my program according to rules mentioned in the assignment pdf:

“Set a signal handler for SIGCHLD in the parent process to handle child process termination.”

“Create a program that takes an integer argument.”

“Create two FIFOs (named pipes).”

“Send an array of random numbers to the first FIFO and the second FIFO. Send a command to the second FIFO(requesting a multiplication operation).”

“If FIFOs cannot be created or if there are errors in data/command transmission, appropriate error messages should be displayed.”

Use the fork() system call to create two child processes and assign each to a FIFO.

All child processes sleep for 10 seconds, execute their tasks, and then exit.

Enter a loop, printing a message containing "proceeding" every two seconds.

and other important rules that are mentioned in pdf.

Printing Proceeding...

```
while(proc_count < 2) {
    printf("Proceeding...\n");
    sleep(2);
}
```

Program Flow:

Parent Process :

- Confirm command line argument
- Generate random numbers
- Create fifos
- Create child processes using fork()
- Open fifos and send them random numbers
- Send “multiply” command to “FIFO2”
- Handle SIGCHLD

Child Process 1 :

- Sleep for 10 seconds.
- Open FIFO1 to read random numbers and calculate sum of them
- Open FIFO2 to write sum to it

Child Process 2 :

- Sleep for 10 seconds.
- Open FIFO2 to get random numbers and calculate product of them
- Read “multiply” command from FIFO2
- Read sum that calculated in child process 1 from FIFO2

Calculate and print Sum of sum and product

Another Output Example:

```
veysel@LAPTOP-28PS531T:~/projects/cse344$ ./gtu 10
Array: 9 0 1 3 4 0 9 2 5 7
Proceeding...
Proceeding...
Proceeding...
Proceeding...
Proceeding...
Array sent to FIFO2
Array sent to FIFO1
Proceeding...
Sum of elements in array at CHLD1: 40
Sum sent to CHLD2: 40
Child with PID 31 terminated. exit status: 0
Proceeding...
Sum from CHLD1 : 40
Result of multiplication: 0
result of summation + result of mult: 40
Child with PID 32 terminated. exit status: 0
Parent exits
```

SUMMARY:

This program demonstrates IPC using FIFOs (named pipes) in a parent-child process setup. Where two child processes perform arithmetic operations on an array of integers provided by the parent process.

The program expects a single command line argument fort he size of the array. It then populates the array with random integers between 0 to 9. The parent process forks two children. The firs child reads the array from FIFO, calculates the sum of elements, and writes the sum back to another FIFO. The second child reads from the second FIFO, expects to receive a command and the array, performs multiplication of the elements, and reads the sum from the first child. It then outputs the results of these operations.

Detailed IPC is handled via two named pipes, FIFO1 AND FIFO2. Error checking is thoroughly implemented to handle issues like failed fork(), failed pipe operations, and more. The parent process tracks the termination of child processes using a signal handler and a process count to ensure it waits for both to complete before it exits.